.NET Data Access
ADO.NET & Entity Framework Core

A Comprehensive Reference for RAG Systems

Covers: ADO.NET Core Components | EF Core ORM | Migrations
LINQ Queries | Relationships | Performance Comparison

Assignment 3 - RAG Chatbot Knowledge Base

1. Introduction to .NET Data Access

.NET provides two primary approaches for interacting with relational databases: ADO.NET and Entity Framework (EF Core).

ADO.NET is the low-level data access technology built into the .NET framework. It gives developers fine-grained control over database interactions using SQL commands and connection objects directly.

Entity Framework Core (EF Core) is a modern Object-Relational Mapper (ORM) that sits on top of ADO.NET. It allows developers to work with databases using .NET objects instead of writing raw SQL, while still using ADO.NET underneath.

Understanding both technologies is essential for any .NET developer - ADO.NET for performance-critical scenarios and EF Core for rapid development and complex domain models.

2. ADO.NET Overview

ADO.NET (ActiveX Data Objects .NET) is the foundational data access library in .NET. It provides a set of classes that expose data access services for .NET Framework applications.

Core Components:

SqlConnection - Represents an open connection to a SQL Server database. You must open it before executing commands and close/dispose it after use. Connection strings specify the server, database, and authentication details.

SqlCommand - Represents a SQL statement or stored procedure to execute against a database. You can set CommandText (the SQL query) and CommandType (Text or StoredProcedure).

SqlDataReader - Provides a fast, forward-only, read-only stream of rows from a database. It is the most efficient way to read large volumes of data.

SqlDataAdapter - Serves as a bridge between a DataSet and a database. It can fill a DataSet with results and later update the database with changes.

DataSet - An in-memory representation of data that can contain multiple DataTable objects. It works disconnected from the database, meaning you can manipulate data offline.

DataTable - Represents one table of in-memory data. It can be populated from a database or created programmatically.

Connection String example:
"Server=localhost;Database=MyDb;User Id=sa;Password=secret;"

Executing Queries:

* ExecuteReader() - returns a SqlDataReader for SELECT queries
* ExecuteScalar() - returns a single value (e.g., COUNT, MAX)
* ExecuteNonQuery() - executes INSERT, UPDATE, DELETE and returns rows affected

3. ADO.NET: Parameterized Queries & Transactions

Parameterized Queries:

Always use parameterized queries to prevent SQL injection attacks. Instead of concatenating user input into SQL strings, use parameter placeholders.

Example pattern:

```
command.CommandText = "SELECT * FROM Users WHERE Email = @Email";
command.Parameters.AddWithValue("@Email", userInput);
```

This ensures the input is treated as data, not executable SQL.

Stored Procedures:

You can call stored procedures by setting CommandType to StoredProcedure and providing the procedure name as CommandText.

Transactions:

ADO.NET supports explicit transactions via SqlTransaction. A transaction groups multiple operations so they either all succeed or all fail together.

Steps:

1. Open a connection
2. Begin a transaction: connection.BeginTransaction()
3. Assign the transaction to each command
4. Call Commit() on success
5. Call Rollback() in the catch block on failure

This ensures data consistency - for example, when transferring money between two accounts, both the debit and credit must succeed or neither happens.

Connection Pooling:

ADO.NET automatically manages a pool of database connections. When you close a connection, it is returned to the pool rather than being destroyed. This improves performance by reusing connections instead of creating new ones each time.

4. Entity Framework Core Overview

Entity Framework Core (EF Core) is an open-source, lightweight, extensible, cross-platform ORM for .NET. It enables developers to work with a database using .NET objects, eliminating most of the data-access code that developers usually need to write.

Key Concepts:

DbContext - The primary class responsible for interacting with the database. It represents a session with the database and can be used to query and save instances of your entities. You configure the database connection in OnConfiguring() and define DbSets in the class body.

DbSet<T> - Represents a collection of entities of a given type in the database, corresponding to a database table. You use it to query, add, update, and delete records.

Entity - A plain C# class (POCO) that maps to a database table. Properties map to columns. Navigation properties represent relationships between tables.

Development Approaches:

Code First - You define your entity classes and DbContext in C#, then EF Core generates the database schema from your code. This is the most popular approach for greenfield projects.

Database First - EF Core reverse-engineers your existing database schema into entity classes and a DbContext using the Scaffold-DbContext command.

Model First - You design a visual model and generate both the database and code from it (less common in EF Core).

5. EF Core: Defining Entities & Relationships

Entities are plain C# classes. EF Core uses conventions and Data Annotations (or Fluent API) to map them to database tables.

Conventions:

* A property named Id or <TypeName>Id becomes the primary key automatically.
* Navigation properties are used to define relationships.
* String properties become nvarchar(max) by default.

Data Annotations:

You can decorate properties with attributes like [Key], [Required], [MaxLength(100)], [Column("col_name")], [Table("tbl_name")] to control mapping behavior.

Fluent API:

In OnModelCreating() inside DbContext, you use modelBuilder to configure entities with more control than annotations allow.

Relationships:

One-to-Many - The most common relationship. One parent entity has many child entities. Defined by a foreign key property and a navigation property on the child, plus a collection navigation on the parent.
Example: One Blog has many Posts. Post has a BlogId foreign key.

One-to-One - One entity relates to exactly one other entity. Both sides have a navigation property pointing to each other.
Example: One User has one UserProfile.

Many-to-Many - Both entities can relate to many of the other. EF Core 5+ handles this automatically with a join table, or you can define an explicit join entity.
Example: Students and Courses - one student takes many courses, one course has many students.

6. EF Core: Migrations

Migrations allow you to evolve your database schema over time as your entity model changes, without losing existing data.

How Migrations Work:

EF Core compares your current entity model against a snapshot of the previous model and generates a migration file containing the changes needed to update the database schema.

Common Commands (using .NET CLI):

dotnet ef migrations add <MigrationName>

* Generates a new migration file based on changes to your entities since the last migration. Always give it a meaningful name like "AddProductTable" or "RenameUserEmail".

dotnet ef database update

* Applies all pending migrations to the database, creating or altering tables as needed.

dotnet ef migrations remove

* Removes the last migration if it has not been applied to the database.

dotnet ef database update <MigrationName>

* Rolls the database forward or backward to a specific migration point.

dotnet ef migrations list

* Lists all migrations and shows which ones have been applied.

Migration Files:

Each migration has two methods:

* Up() - contains the changes to apply (create table, add column, etc.)
* Down() - contains the changes to reverse (drop table, drop column, etc.)

A __EFMigrationsHistory table is created in the database to track which migrations have been applied.

Best Practices:

* Always review generated migration files before applying them.
* Never edit migration files that have already been applied in production.
* Use meaningful migration names.
* Keep migrations small and focused.

7. EF Core: Querying Data with LINQ

EF Core translates LINQ (Language Integrated Query) expressions into SQL queries automatically.

Basic Query Patterns:

ToList() - Executes the query and returns all matching rows as a list.
Example: dbContext.Products.ToList()

Where() - Filters rows based on a condition.
Example: dbContext.Products.Where(p => p.Price > 100).ToList()

FirstOrDefault() - Returns the first matching record or null.
Example: dbContext.Users.FirstOrDefault(u => u.Email == email)

Find() - Looks up an entity by its primary key. Checks the change tracker first before going to the database.

OrderBy() / OrderByDescending() - Sort results.

Select() - Projects results into a different shape (e.g., anonymous object or DTO).

Count() / Any() / Sum() / Max() / Min() - Aggregate operations.

Loading Related Data:

Eager Loading - Load related entities in the same query using Include().
Example: dbContext.Blogs.Include(b => b.Posts).ToList()
Use ThenInclude() to load nested relationships.

Lazy Loading - Related data is loaded automatically when a navigation property is first accessed. Requires virtual navigation properties and the Microsoft.EntityFrameworkCore.Proxies package.

Explicit Loading - Manually load related data on demand using Load().
Example: dbContext.Entry(blog).Collection(b => b.Posts).Load()

No-Tracking Queries:
Use AsNoTracking() for read-only queries where you do not intend to update the data. This improves performance by skipping the overhead of the change tracker.
Example: dbContext.Products.AsNoTracking().ToList()

8. EF Core: Saving Data

EF Core tracks changes to entities loaded from the database. When you call SaveChanges(), it generates the appropriate INSERT, UPDATE, or DELETE SQL and executes it.

Adding Data:

Use dbContext.Add(entity) or dbContext.Set<T>().Add(entity) to mark a new entity for insertion. Call SaveChanges() to execute the INSERT.

Updating Data:

If you load an entity from the database, modify its properties, and call SaveChanges(), EF Core detects the changes and generates an UPDATE statement for only the changed columns.

If you have a detached entity (not loaded from the current context), use dbContext.Update(entity) to mark it as modified.

Deleting Data:

Use dbContext.Remove(entity) to mark an entity for deletion. Call SaveChanges() to execute the DELETE.

SaveChanges():

Wraps all pending operations in a single database transaction by default. If any operation fails, none are applied.

SaveChangesAsync():

The asynchronous version. Preferred in web applications to avoid blocking threads while the database operation completes.

Change Tracker:

EF Core's change tracker monitors entity states: Added, Modified, Deleted, Unchanged, Detached. You can inspect entity states via dbContext.Entry(entity).State.

Bulk Operations:

For inserting or updating thousands of records, SaveChanges() in a loop is inefficient. Consider using libraries like EFCore.BulkExtensions for bulk inserts/updates.

Concurrency Handling:

EF Core supports optimistic concurrency. Add a [Timestamp] or [ConcurrencyCheck] attribute to a property, and EF Core will detect if another user changed the record since you loaded it, throwing a DbUpdateConcurrencyException.

9. ADO.NET vs Entity Framework Core - Comparison

Both ADO.NET and EF Core have their place in .NET development. Choosing between them depends on the use case.

Performance:

ADO.NET is faster because it has no ORM overhead. You write direct SQL that runs exactly as intended. EF Core adds a layer of abstraction - LINQ translation, change tracking, and proxy generation - which introduces some overhead. However, with AsNoTracking() and compiled queries, EF Core performance can get very close to ADO.NET for read-heavy scenarios.

Development Speed:

EF Core wins for development speed. You write less code, get automatic CRUD operations, and avoid writing repetitive SQL boilerplate. ADO.NET requires manual SQL for every operation.

Control:

ADO.NET gives you full control over every SQL statement. EF Core generates SQL for you, which may not always be optimal. You can inspect generated SQL using logging and override it with raw SQL when needed.

Complexity:

ADO.NET is conceptually simpler - it is just wrappers around database connections and commands. EF Core has a steeper learning curve due to migrations, relationships, change tracking, and lazy/eager loading concepts.

When to Use ADO.NET:

* High-performance, read-heavy applications
* Complex stored procedures with custom SQL
* Bulk insert/update operations
* Scenarios where you need byte-level control over queries

When to Use EF Core:

* Rapid application development
* Domain-driven design with complex object graphs
* Applications where developer productivity matters more than raw performance
* CRUD-heavy business applications
* When you want automatic schema management via migrations

Comparison Table:

Feature	ADO.NET	EF Core
Abstraction Level	Low	High
SQL Control	Full	Partial (raw SQL available)
Performance	Fastest	Near ADO.NET with tuning
Learning Curve	Low	Moderate
Migrations	Manual	Automatic
Change Tracking	Manual	Automatic
Relationships	Manual joins	Navigation properties

LINQ Support	No	Yes
Best For	Performance	Productivity

10. Code Patterns Reference

ADO.NET - Basic Query Pattern:

```
using var connection = new SqlConnection(connectionString);
connection.Open();
using var command = new SqlCommand("SELECT Id, Name FROM Products WHERE Price > @Price", connection);
command.Parameters.AddWithValue("@Price", 50);
using var reader = command.ExecuteReader();
while (reader.Read())
{
int id = reader.GetInt32(0);
string name = reader.GetString(1);
}
```

ADO.NET - Insert with Transaction:

```
using var connection = new SqlConnection(connectionString);
connection.Open();
using var transaction = connection.BeginTransaction();
try
{
var cmd = new SqlCommand("INSERT INTO Orders (CustomerId, Total) VALUES (@CId, @Total)", connection, transaction);
cmd.Parameters.AddWithValue("@CId", customerId);
cmd.Parameters.AddWithValue("@Total", total);
cmd.ExecuteNonQuery();
transaction.Commit();
}
catch
{
transaction.Rollback();
throw;
}
```

EF Core - Define Entity and Context:

```
public class Product
{
public int Id { get; set; }
public string Name { get; set; }
public decimal Price { get; set; }
}

public class AppDbContext : DbContext
```

```
{
public DbSet<Product> Products { get; set; }
protected override void OnConfiguring(DbContextOptionsBuilder options)
=> options.UseSqlServer(connectionString);
}
```

EF Core - Query and Save:

```
using var db = new AppDbContext();

// Query
var expensiveProducts = db.Products
.Where(p => p.Price > 100)
.OrderBy(p => p.Name)
.AsNoTracking()
.ToList();

// Insert
db.Products.Add(new Product { Name = "Widget", Price = 29.99m });
db.SaveChanges();

// Update
var product = db.Products.Find(1);
product.Price = 39.99m;
db.SaveChanges();

// Delete
var toDelete = db.Products.Find(2);
db.Products.Remove(toDelete);
db.SaveChanges();
```

EF Core - Eager Loading with Include:

```
var blogs = db.Blogs
.Include(b => b.Posts)
.ThenInclude(p => p.Comments)
.ToList();
```

11. Summary

ADO.NET is the foundation of .NET data access - a thin, fast, explicit layer over database connections and SQL commands. It is ideal when performance is paramount or when working with complex, hand-tuned SQL.

Entity Framework Core is the modern, productive way to build data-driven .NET applications. It abstracts away repetitive SQL, manages schema evolution through migrations, and allows you to model your domain naturally using C# classes and relationships.

In practice, many applications use both: EF Core for the majority of CRUD operations and ADO.NET (or raw SQL via EF Core's FromSqlRaw) for performance-critical queries or bulk operations.

Key Takeaways:

* Use ADO.NET when you need maximum performance and control.
* Use EF Core when developer productivity and maintainability are the priority.
* Parameterized queries (both ADO.NET and EF Core) are essential for security.
* EF Core migrations make database schema management much safer and repeatable.
* AsNoTracking() is an easy performance win for read-only EF Core queries.
* Transactions ensure data consistency across multiple operations in both approaches.